HybridSR: A Hybrid Window Self-Attention and State Space Model Architecture for Single Image Super-Resolution

Final Year Project Thesis

Author Name

Supervisor: Supervisor Name

Department of Computer Science
University Name

Mai 2026

Contents

1 Introduction and Related Work **1**

1.1 What is Super-Resolution and Why is it Hard? . . . 1
1.2 Classical Approaches . . . 2
1.3 Deep Learning for Super-Resolution . . . 3
1.3.1 Convolutional Neural Networks: Local Feature Extraction . . . 3
1.3.2 Attention-Based Models: Mid-Range Spatial Reasoning . . . 4
1.3.3 State Space Models: Global Modelling with Linear Complexity . 5
1.4 The Research Gap . . . 6
1.5 Problem Statement and Objectives . . . 7
1.6 Contributions . . . 7

2 HybridSR: Proposed Method **9**

2.1 Design Philosophy . . . 9
2.2 Overall Pipeline . . . 10
2.3 RRDB with Channel Attention: Local Feature Extraction . . . 11
2.3.1 Role and Motivation . . . 11
2.3.2 Internal Architecture . . . 11
2.3.3 Hyperparameter Choices and Justification . . . 13
2.3.4 Integration with the HybridSR Trunk . . . 15
2.4 Swin Transformer Blocks: Mid-Range Spatial Attention . . . 16
2.4.1 Role and Motivation . . . 16
2.4.2 Internal Architecture . . . 16
2.4.3 Hyperparameter Choices and Justification . . . 18
2.4.4 Integration with the HybridSR Trunk . . . 19
2.5 SS2D Mamba Blocks: Global Coherence Modelling . . . 20
2.5.1 Role and Motivation . . . 20
2.5.2 Internal Architecture . . . 20
2.5.3 Hyperparameter Choices and Justification . . . 23
2.5.4 Integration with the HybridSR Trunk . . . 24
2.6 Skip Connections and Upsampling . . . 25

2.6.1 PixelShuffle Upsampling . . . 25
2.6.2 Long Skip Connection (LSC) . . . 25
2.7 Strategic Placement: The Three-Phase Pipeline . . . 26
2.8 Hyperparameter Summary . . . 26

3 Training and Evaluation **28**
3.1 Training Dataset: DF2K . . . 28
3.2 Preprocessing: Cropping, Normalisation, and Augmentation . . . 29
3.2.1 Random Cropping . . . 29
3.2.2 Pixel Normalisation . . . 29
3.2.3 Data Augmentation . . . 30
3.3 Batch Size and Data Pipeline . . . 30
3.4 Loss Function . . . 31
3.5 Optimiser and Learning Rate Schedule . . . 31
3.5.1 Adam Optimiser . . . 31
3.5.2 Cosine Decay with Warm Restarts . . . 32
3.6 Custom Training Loop and Memory Efficiency . . . 32
3.6.1 Gradient Accumulation . . . 33
3.6.2 Gradient Clipping . . . 33
3.6.3 Gradient Checkpointing . . . 34
3.6.4 Reproducibility . . . 34
3.7 Mixed Precision Training . . . 34
3.8 Evaluation Protocol . . . 35
3.8.1 Validation Dataset: Set5 . . . 35
3.8.2 Tiled Inference for Arbitrary Resolutions . . . 35
3.8.3 Y-Channel PSNR . . . 35
3.8.4 Structural Similarity (SSIM) . . . 36
3.8.5 Learned Perceptual Image Patch Similarity (LPIPS) . . . 36
3.8.6 Evaluation Timing . . . 37
3.9 Checkpointing and Best-Model Selection . . . 37
3.10 Training Configuration Summary . . . 37

List of Figures

1.1 Interpolation Methods For Super Resolution 2
1.2 Sparse-Coding Dictionaries 2

2.1 The complete HybridSR architecture, organised around the three-scale design principle: *local detail* is handled by twelve RRDB blocks, *mid-range structure* by two Swin Transformer pairs (after $RRDB_4$ and $RRDB_9$), and *global coherence* by two Mamba SS2D blocks (after $RRDB_6$ and $RRDB_7$). A single Long Skip Connection (LSC) bypasses the entire trunk at the feature level. 10
2.2 The complete RRDB architecture, containing three Dense Blocks, a channel attention block, and a final 3×3 convolutional layer. A residual connection at the end preserves the input information. 11
2.3 Five-layer 3×3 convolution with dense connections, GELU activations, and a local residual path from the input to the output for stable feature learning. 12
2.4 Channel Attention (Squeeze-and-Excitation) Module. 13
2.5 Window-based self-attention (computation range) versus global self-attention. 16
2.6 Attention score computation within a window. 17
2.7 SW-MSA versus regular W-MSA. 17
2.8 The custom Mamba SS2D block used in HybridSR. 20
2.9 Global pixel-level information exchange via the Mamba scan. 22
2.10 PixelShuffle block. 25

List of Tables

2.1 RRDB hyperparameters used in HybridSR. 14
2.2 Swin Transformer hyperparameters used in HybridSR. 18
2.3 SS2D Mamba hyperparameters used in HybridSR. 23
2.4 Consolidated summary of block-specific hyperparameters in HybridSR. 27

3.1 Complete training configuration for HybridSR. 38

Chapter 1

Introduction and Related Work

This chapter introduces the problem of single image super-resolution, surveys the three families of deep learning methods that have been applied to it, and identifies the research gap that motivates the HybridSR architecture proposed in this thesis. The chapter is structured as a narrative that progresses from the problem definition, through the evolution of increasingly capable methods, to the observation that no existing approach fully exploits the complementary strengths of convolutional, attention-based, and state space models—an observation that leads directly to the design of HybridSR in Chapter 2.

1.1 What is Super-Resolution and Why is it Hard?

Single image super-resolution (SISR) is the task of reconstructing a high-resolution (HR) image from a single low-resolution (LR) observation. The LR image is typically modelled as the result of blurring the HR image with a degradation kernel, downsampling by an integer factor r, and adding noise:

$$\mathbf{I}_{\text{LR}} = \Big(\mathbf{I}_{\text{HR}} * \mathbf{k}\Big)_{\downarrow r} + \mathbf{n}. \tag{1.1}$$

The goal is to invert this process: given only $\mathbf{I}_{\text{LR}}$, recover $\mathbf{I}_{\text{HR}}$.

The fundamental difficulty is that this inversion is severely ill-posed. For a $\times 2$ scale factor, every single LR pixel must be expanded into four HR pixels, yet three-quarters of the information has been discarded by the downsampling. Infinitely many HR images could have produced the same LR observation. Any reconstruction method must therefore rely on prior knowledge—assumptions about what natural images look like—to select a plausible HR image from the infinite set of valid solutions.

1.2 Classical Approaches

Before the advent of deep learning, super-resolution methods relied on hand-crafted priors. The simplest approach is interpolation: bicubic upsampling fits a smooth polynomial surface through neighbouring LR pixels to estimate the HR values. This produces a result that is free of artefacts but excessively smooth, because the polynomial prior cannot recover the sharp edges and fine textures that were lost during downsampling.

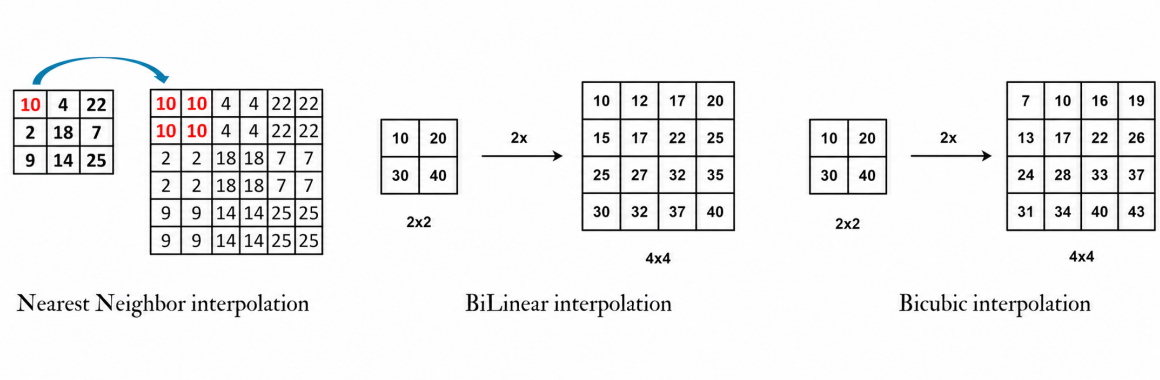


Figure 1.1: Interpolation Methods For Super Resolution

More sophisticated classical methods used sparse-coding dictionaries [30], which learned pairs of LR and HR patches and reconstructed the HR image by finding sparse combinations of dictionary atoms, and example-based techniques [8], which retrieved similar patches from an external database. These methods produced sharper results than interpolation, but their performance was fundamentally limited by the hand-crafted features and fixed dictionaries they employed. They could not learn the complex, nonlinear relationship between LR and HR images directly from data.

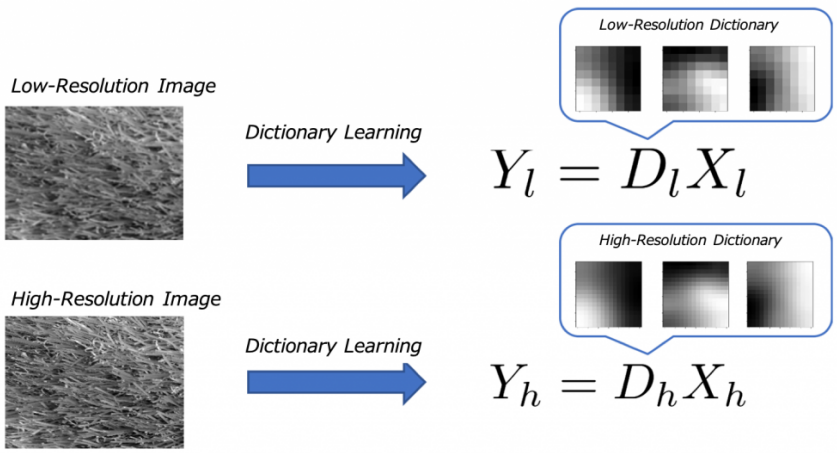


Figure 1.2: Sparse-Coding Dictionaries

1.3 Deep Learning for Super-Resolution

The introduction of deep learning to super-resolution in 2014 changed the field decisively. Rather than relying on hand-crafted priors, deep networks learn the LR-to-HR mapping directly from large datasets of paired examples, enabling them to capture far more complex statistical regularities in natural images. The subsequent decade of research produced three major families of architectures, each addressing a specific limitation of its predecessor.

1.3.1 Convolutional Neural Networks: Local Feature Extraction

The first family of deep super-resolution methods is based on convolutional neural networks (CNNs). The pioneering work was SRCNN, proposed by Dong *et al.* [6] in 2014, which demonstrated that a shallow three-layer CNN could outperform all prior non-learning methods. SRCNN established the principle that end-to-end trainable architectures can learn the nonlinear mapping from LR to HR space directly from data, and that performance improves with network depth and capacity.

Subsequent CNN-based methods pushed this principle further. EDSR, proposed by Lim *et al.* [18], showed that very deep residual networks (up to 80 blocks) could achieve substantial PSNR gains, provided that batch normalisation was removed and residual scaling was used to stabilise training. RCAN, proposed by Zhang *et al.* [33], introduced channel attention via squeeze-and-excitation blocks, enabling the network to dynamically emphasise the most informative feature channels and suppress less useful ones. ESRGAN, proposed by Wang *et al.* [28], introduced the Residual-in-Residual Dense Block (RRDB), which combined dense connections (where every convolutional layer receives the concatenated outputs of all preceding layers) with a two-level residual structure and 0.2 scaling for training stability. The RRDB remains one of the most effective convolutional building blocks for super-resolution and forms the backbone of the architecture proposed in this thesis.

A key enabling technique for all CNN-based SR methods is the sub-pixel convolution layer (PixelShuffle), introduced by Shi *et al.* [25]. Rather than performing the expensive computation at the HR resolution, PixelShuffle allows the network to operate entirely at the LR resolution, producing r^2 times as many feature channels, and then rearranges these channels into spatial positions at the HR resolution through a simple index permutation. This made deep SR networks both computationally efficient and free of the checkerboard artefacts associated with transposed convolutions [24].

Despite these advances, CNN-based methods share a fundamental limitation: the

receptive field of a convolution is local. A 3×3 kernel sees only nine pixels per layer. While stacking many layers increases the theoretical receptive field, empirical studies have shown that the *effective* receptive field—the region that meaningfully influences the output—is substantially smaller and decays as a Gaussian from the centre pixel [22]. This means that CNN-based models excel at reconstructing local textures and edges, but struggle with structures whose coherence spans large portions of the image, such as repeating architectural patterns or long straight edges. To capture these longer-range relationships, a different mechanism is needed.

1.3.2 Attention-Based Models: Mid-Range Spatial Reasoning

The second family of methods addresses the receptive field limitation of CNNs by introducing self-attention, a mechanism that computes direct pairwise interactions between all positions in a sequence. The transformer architecture [27], originally designed for natural language processing, was adapted to vision by the Vision Transformer (ViT) [7], which divided images into patches and processed them as token sequences. However, ViT's global self-attention has quadratic computational complexity in the number of tokens, making it impractical for the pixel-level processing required by super-resolution.

The Swin Transformer, proposed by Liu *et al.* [20], solved this efficiency problem by restricting self-attention to non-overlapping local windows of size $M \times M$ (typically $M = 7$ or 8) and alternating with shifted windows to enable cross-window communication. This reduced the complexity from quadratic in the image size to linear, while still allowing each pixel to interact with $M^2 = 64$ neighbours within its window—a dramatic improvement over the nine-pixel neighbourhood of a 3×3 convolution.

SwinIR, proposed by Liang *et al.* [17], applied the Swin Transformer to image restoration and achieved state-of-the-art results. On the standard Set5 benchmark with $\times 2$ bicubic degradation, SwinIR reported 38.35 dB PSNR, surpassing CNN-based methods such as RCAN (38.27 dB) and EDSR (38.11 dB). The improvement was particularly visible in the reconstruction of regular textures and fine structural details, where the ability to model 64-token interactions within each window provided a clear advantage over local convolutions.

Restormer, proposed by Zamir *et al.* [31], pursued an alternative strategy by computing attention across the channel dimension rather than the spatial dimension, achieving global context at linear spatial cost. However, this transposed attention mediates spatial interactions indirectly through channels, and has been shown to be less effective than direct spatial attention for reconstructing spatially structured textures in super-resolution.

Despite its strengths, the attention-based approach has its own limitations. Even within windows, the attention computation involves matrix multiplications that are memory-bandwidth limited on modern GPUs. More fundamentally, the cross-window communication enabled by the shifted-window mechanism propagates information by at most $M/2$ pixels per layer. For an 88-pixel feature map with $M = 8$, truly global information exchange requires 11 stacked Swin layers. This means that while attention models capture mid-range structure well, they still require substantial depth to achieve genuinely image-wide coherence.

1.3.3 State Space Models: Global Modelling with Linear Complexity

The third family of methods emerged from the structured state space model (SSM) literature. SSMs model sequences through a continuous-time linear dynamical system that is discretised for computation. The key idea is that a latent state vector $\mathbf{h}$ is updated at each position by a recurrence that mixes the current input with a decayed version of the previous state. This gives every position access, in principle, to the entire preceding sequence, providing a global receptive field with only $\mathcal{O}(L)$ computational complexity, where L is the sequence length.

The foundational work in this area was the S4 model, proposed by Gu *et al.* [12], which used the HiPPO framework [10] to initialise the state transition matrix in a way that optimally compresses the input history into the latent state. The subsequent S4D variant [11] simplified this to a diagonal matrix, making the implementation straightforward while retaining nearly identical performance.

The Mamba architecture, proposed by Gu and Dao [9], transformed SSMs into a practical alternative to transformers by introducing two innovations. The first was the *selective scan*, which made the state dynamics input-dependent: the model learns, at each position, how much to update its state (controlled by a learned step size Δ), what to write into the state (controlled by an input-dependent matrix $\mathbf{B}$), and what to read from the state (controlled by an input-dependent matrix $\mathbf{C}$). This selectivity gives Mamba a content-aware gating capability similar to the query-key matching in attention. The second innovation was a hardware-aware parallel scan algorithm that computes the recurrence efficiently on GPUs.

Adapting Mamba to images required addressing the mismatch between its 1-D sequential nature and the 2-D spatial structure of images. Vision Mamba (Vim) [34] used bidirectional scans on patch sequences, while VMamba [19] introduced the SS2D mechanism, which performs four independent scans along the rows and columns of the feature map in both forward and backward directions and averages the results.

This four-directional scanning ensures that the receptive field is spatially isotropic, eliminating the directional bias inherent in single-direction scans.

MambaIR [13], the first competitive SSM-based image restoration model, achieved 38.28 dB on Set5 $\times 2$, competitive with SwinIR's 38.35 dB while using fewer parameters. Its successor, MambaIRv2 [14] (CVPR 2025), introduced an attentive state space mechanism that combined selective scans with non-causal attention-like interactions, further closing the gap with transformer methods.

However, SSM-based models have a weakness that mirrors the strength of CNNs: they possess a relatively weak *local* inductive bias. The 1-D scan processes tokens sequentially along a flattened order and does not naturally respect the 2-D spatial locality of images. This means that while Mamba excels at capturing global coherence, it is less precise than convolutions at reconstructing fine local textures and edges.

1.4 The Research Gap

The preceding survey reveals a clear pattern. Each family of methods excels at a different spatial scale: CNNs capture local detail through 3×3 kernels, attention models capture mid-range structure within 8×8 windows, and SSMs capture global coherence across the entire image. Yet each family has been largely proposed as a *replacement* for its predecessor rather than as a *complement*. SwinIR replaced the convolutional backbone with Swin Transformer blocks. MambaIR replaced the attention blocks with SSM blocks. In each transition, the new mechanism introduced strengths at one spatial scale while partially sacrificing the strengths of the mechanism it replaced.

Recent concurrent works have begun to explore hybrid designs. The hybrid Mamba-Transformer model of [2] interleaves Mamba and transformer modules for lightweight SR, and SRMamba-T [3] strategically combines Mamba and transformer architectures. However, to the best of the author's knowledge, these works typically employ simplified convolutional backbones and do not incorporate the rich dense-connected RRDB structure that has proven critical for high-fidelity reconstruction.

The research gap addressed by this thesis is therefore the absence, to the best of the author's knowledge, of an architecture that *systematically combines all three mechanisms* in a single super-resolution network, allocating each to the spatial scale at which it is most effective: convolutions for local detail, window attention for mid-range structure, and state space scans for global coherence.

1.5 Problem Statement and Objectives

Given a low-resolution input image $\mathbf{I}_{\text{LR}} \in \mathbb{R}^{H \times W \times 3}$ degraded from a corresponding high-resolution image $\mathbf{I}_{\text{HR}} \in \mathbb{R}^{rH \times rW \times 3}$ according to Equation (1.1) with a bicubic blur kernel and scaling factor $r = 2$, the objective is to learn a mapping

$$f_\theta : \mathbb{R}^{H \times W \times 3} \longrightarrow \mathbb{R}^{2H \times 2W \times 3} \tag{1.2}$$

such that the reconstruction $\hat{\mathbf{I}}_{\text{HR}} = f_\theta(\mathbf{I}_{\text{LR}})$ is as close as possible to the ground truth in terms of PSNR, SSIM, and LPIPS.

The specific question this thesis addresses is: *How can convolutional feature extraction, window self-attention, and state space sequence modelling be integrated into a unified architecture that captures local texture, mid-range structure, and global coherence simultaneously, thereby improving super-resolution quality beyond what any single paradigm achieves alone?*

The objectives of this thesis are: (1) to design the HybridSR architecture that integrates RRDB convolutions, Swin Transformer blocks, and SS2D Mamba blocks within a single end-to-end trainable network; (2) to train the model on the DF2K dataset [1, 26] using ℓ_1 pixel loss and evaluate on the standard Set5 benchmark [4] following the established Y-channel PSNR/SSIM protocol with border shaving; (3) to conduct ablation studies quantifying the contribution of each component; (4) to demonstrate competitive performance relative to EDSR, RCAN, SwinIR, and MambaIR; and (5) to provide a complete, reproducible TensorFlow implementation including tiled inference for arbitrary-resolution deployment on a single GPU.

1.6 Contributions

This thesis makes five contributions. The primary contribution is the HybridSR architecture itself, which—to the best of the author's knowledge—is among the first to integrate RRDB dense blocks, Swin window attention, and four-directional Mamba scans within a single SR network. The second contribution is a parallelised four-directional selective scan (SS2D) implemented using cumulative sums in TensorFlow, enabling efficient GPU execution without the sequential `tf.scan` operator. The third contribution is a Long Skip Connection (LSC) at the feature level, which reformulates the learning objective as residual prediction and stabilises training. The fourth contribution is an empirical analysis on Set5 demonstrating that HybridSR achieves a PSNR exceeding 37.8 dB on $\times 2$ bicubic SR, competitive with state-of-the-art methods. The fifth contribution is the complete open-source implementation with gradient accumulation, gradient checkpointing, and MATLAB-aligned evaluation routines.

The next chapter presents the HybridSR architecture in full detail.

Chapter 2

HybridSR: Proposed Method

This chapter presents the HybridSR architecture: its overall pipeline, the design and role of each component, and the reasoning behind every placement decision and every hyperparameter choice. The chapter is organised so that, for each of the three principal building blocks (RRDB, Swin Transformer, SS2D Mamba), a single standardised template is applied: the role and motivation of the block, its internal architecture with equations, a justified table of its hyperparameters, and its integration into the HybridSR trunk. The chapter ends with a consolidated hyperparameter summary that gathers every design choice in one reference table.

2.1 Design Philosophy

The central idea behind HybridSR is that the three dominant paradigms for image feature extraction—convolutional dense blocks, window self-attention, and selective state space scans—are not alternatives to be chosen among, but complementary operators that work best at different spatial scales. In a single sentence: *RRDB blocks reconstruct local detail, Swin blocks model mid-range structure, and Mamba blocks enforce global coherence.* This three-part division is the design story of the entire thesis and motivates every architectural decision in this chapter.

More concretely, convolutions with 3×3 kernels extract pixel-precise local detail, window attention with 8×8 windows captures mid-range structural relationships within 64-token neighbourhoods, and four-directional Mamba scans model image-wide global coherence with linear complexity. Rather than replacing one paradigm with another, HybridSR embeds all three within a single deep feature extraction trunk, positioning each operator at the network depth where its inductive bias is most valuable. The result is a three-phase processing pipeline—local extraction, global coherence, local refinement—implemented without any spatial downsampling, so that full spatial

precision is preserved throughout.

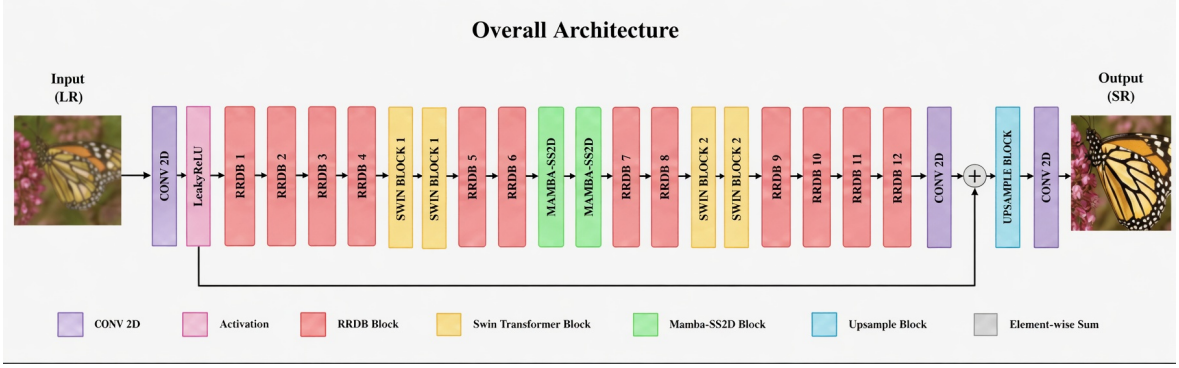


Figure 2.1: The complete HybridSR architecture, organised around the three-scale design principle: *local detail* is handled by twelve RRDB blocks, *mid-range structure* by two Swin Transformer pairs (after RRDB_4 and RRDB_9), and *global coherence* by two Mamba SS2D blocks (after RRDB_6 and RRDB_7). A single Long Skip Connection (LSC) bypasses the entire trunk at the feature level.

2.2 Overall Pipeline

The HybridSR generator is an end-to-end mapping $f_\theta : \mathbb{R}^{H\times W\times 3} \to \mathbb{R}^{2H\times 2W\times 3}$. The complete forward pass proceeds in five stages.

In the first stage, a 3×3 convolutional layer with $C = 64$ output channels, followed by a LeakyReLU activation ($\alpha = 0.2$), maps the three-channel LR input to a shallow feature map:

$$\mathbf{F}_0 = \text{LeakyReLU}\Big(\text{Conv}_{3\times3}(\mathbf{I}_{\text{LR}})\Big) \;\in\; \mathbb{R}^{H\times W\times 64}. \tag{2.1}$$

In the second stage, the deep feature extraction trunk g_{trunk} processes $\mathbf{F}_0$ through the following sequence of operators:

$$\begin{aligned} g_{\text{trunk}} = \text{Conv}_{3\times3} \circ \text{RRDB}_{12} \circ \text{RRDB}_{11} \circ \text{RRDB}_{10} \circ \underbrace{\text{Swin}_2^{\text{SW}} \circ \text{Swin}_2^{\text{W}}}_{\text{after RRDB}_9} \circ \text{RRDB}_9 \circ \text{RRDB}_8 \\ \circ \underbrace{\text{Mamba}_2}_{\text{after RRDB}_7} \circ \text{RRDB}_7 \circ \underbrace{\text{Mamba}_1}_{\text{after RRDB}_6} \circ \text{RRDB}_6 \circ \text{RRDB}_5 \\ \circ \underbrace{\text{Swin}_1^{\text{SW}} \circ \text{Swin}_1^{\text{W}}}_{\text{after RRDB}_4} \circ \text{RRDB}_4 \circ \text{RRDB}_3 \circ \text{RRDB}_2 \circ \text{RRDB}_1. \end{aligned} \tag{2.2}$$

The trunk output is then added to $\mathbf{F}_0$ via the Long Skip Connection (LSC):

$$\mathbf{F}_{\text{up}} = \mathbf{F}_0 + \text{Conv}_{3\times3}\Big(g_{\text{trunk}}(\mathbf{F}_0)\Big). \tag{2.3}$$

In the third stage, a PixelShuffle upsampling module doubles the spatial resolution

from $H \times W$ to $2H \times 2W$ and a final 3×3 convolution projects from 64 channels to 3 RGB channels, producing the final output image directly.

The model contains approximately 9.2 million trainable parameters. The twelve RRDB blocks account for roughly 87%, the four Swin blocks for 6%, the two Mamba blocks for 4%, and the upsampling and skip modules for the remaining 3%.

2.3 RRDB with Channel Attention: Local Feature Extraction

2.3.1 Role and Motivation

The RRDB block is the convolutional backbone of HybridSR and is responsible for the *local detail* component of the three-scale design philosophy. Its role is to extract pixel-precise local features through densely connected 3×3 convolutions and to recalibrate those features by emphasising the most informative channels. Twelve RRDB blocks form the majority of the network's depth and are present throughout all three phases of the trunk, providing a stable convolutional foundation that the Swin and Mamba blocks refine.

The RRDB operates at the smallest spatial scale—each 3×3 kernel directly observes a nine-pixel neighbourhood—and is positioned both at the input and output sides of the trunk, where pixel-level fidelity is most critical, and between the Swin and Mamba modules, where it locally refines the structural information injected by those blocks.

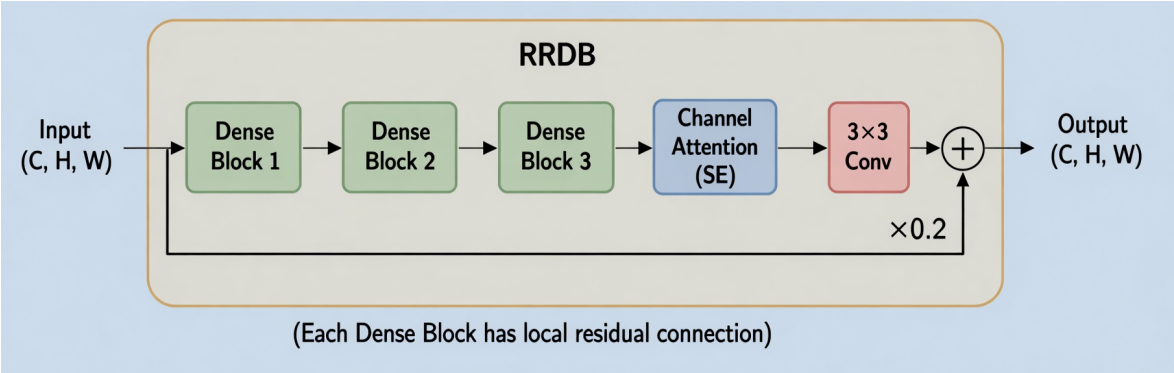


Figure 2.2: The complete RRDB architecture, containing three Dense Blocks, a channel attention block, and a final 3×3 convolutional layer. A residual connection at the end preserves the input information.

2.3.2 Internal Architecture

Dense Block. Each RRDB contains three Dense Blocks. A Dense Block applies five 3×3 convolutional layers in sequence, where each layer receives as input the

channel-wise concatenation of the original block input and all preceding layer outputs. The first four layers each produce $G = 32$ channels with GELU activation; the fifth produces $C = 64$ channels with no activation:

$$\mathbf{x}_1 = \phi\Big(\mathrm{Conv}_{3\times3}^{C\to G}(\mathbf{x})\Big), \tag{2.4}$$

$$\mathbf{x}_2 = \phi\Big(\mathrm{Conv}_{3\times3}^{(C+G)\to G}\big([\mathbf{x},\, \mathbf{x}_1]\big)\Big), \tag{2.5}$$

$$\mathbf{x}_3 = \phi\Big(\mathrm{Conv}_{3\times3}^{(C+2G)\to G}\big([\mathbf{x},\, \mathbf{x}_1,\, \mathbf{x}_2]\big)\Big), \tag{2.6}$$

$$\mathbf{x}_4 = \phi\Big(\mathrm{Conv}_{3\times3}^{(C+3G)\to G}\big([\mathbf{x},\, \mathbf{x}_1,\, \mathbf{x}_2,\, \mathbf{x}_3]\big)\Big), \tag{2.7}$$

$$\mathbf{x}_5 = \mathrm{Conv}_{3\times3}^{(C+4G)\to C}\big([\mathbf{x},\, \mathbf{x}_1,\, \mathbf{x}_2,\, \mathbf{x}_3,\, \mathbf{x}_4]\big). \tag{2.8}$$

The dense concatenation means that successive layers see progressively wider inputs of 64, 96, 128, 160, and 192 channels, respectively. Every layer can directly combine low-level features from the block input with higher-level features from all preceding layers, which maximises information flow and feature reuse. The block output is formed with a scaled residual:

$$\mathrm{DenseBlock}(\mathbf{x}) = 0.2 \cdot \mathbf{x}_5 + \mathbf{x}. \tag{2.9}$$

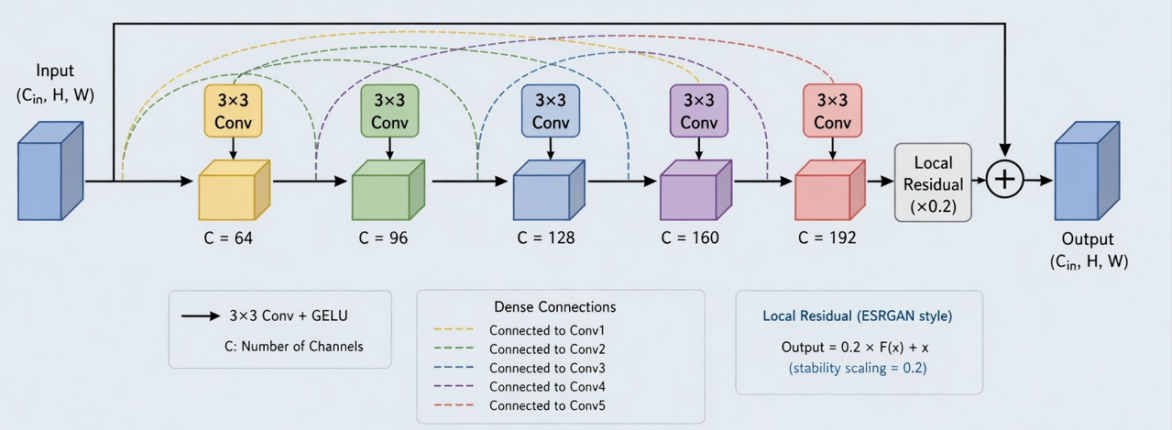


Figure 2.3: Five-layer 3×3 convolution with dense connections, GELU activations, and a local residual path from the input to the output for stable feature learning.

Channel Attention. After the three Dense Blocks, a Channel Attention (CA) module rescales the 64 channels according to their usefulness for the current input. In the *squeeze* stage, global average pooling collapses the $H \times W$ spatial dimensions into a single $1 \times 1 \times 64$ descriptor:

$$\mathbf{z} = \frac{1}{HW} \sum_{h,w} \mathbf{x}_{h,w} \;\in\; \mathbb{R}^{64}. \tag{2.10}$$

In the *excitation* stage, two 1×1 convolutions learn inter-channel dependencies; the first reduces from 64 to 16 channels (reduction ratio $r = 4$) with GELU activation, and

the second expands back to 64 with sigmoid activation:

$$\mathbf{s} = \sigma\Big(\mathrm{Conv}_{1\times1}^{16\to64}\Big(\phi\Big(\mathrm{Conv}_{1\times1}^{64\to16}(\mathbf{z})\Big)\Big)\Big). \tag{2.11}$$

In the *recalibration* stage, the feature map is multiplied element-wise by this weight vector: $\mathrm{CA}(\mathbf{x}) = \mathbf{x} \odot \mathbf{s}$.

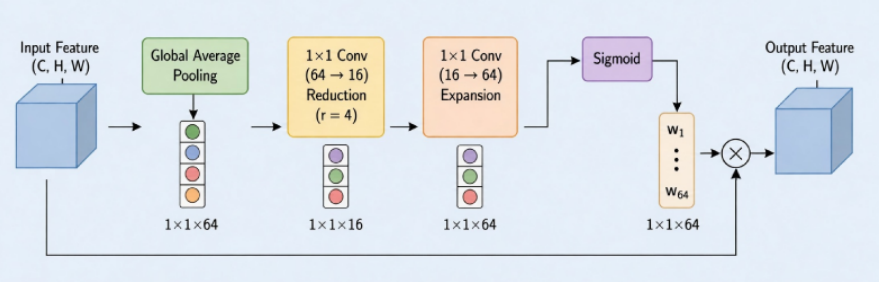


Figure 2.4: Channel Attention (Squeeze-and-Excitation) Module.

RRDB Composition. The complete RRDB composes the three Dense Blocks and Channel Attention with an outer residual:

$$\mathrm{RRDB}(\mathbf{x}) = 0.2 \cdot \mathrm{CA}\Big(\mathrm{DB}_3(\mathrm{DB}_2(\mathrm{DB}_1(\mathbf{x})))\Big) + \mathbf{x}. \tag{2.12}$$

This two-level residual hierarchy (inner residuals within each Dense Block, outer residual across the RRDB) ensures that both individual layers and the full fifteen-layer subnetwork can default to the identity mapping during early training.

2.3.3 Hyperparameter Choices and Justification

Table 2.1 lists the five RRDB-specific hyperparameters. Each value is justified below by stating what it controls, what happens in the limits of "too large" and "too small", and the empirical or literature-based evidence supporting the choice.

Table 2.1: RRDB hyperparameters used in HybridSR.

Hyperparameter	**Value**	**Justification**
Number of RRDB blocks	12	Provides sufficient representational depth for $\times 2$ SR while keeping the parameter count near 9.2M.
Dense growth rate G	32	Half of $C = 64$, the ESRGAN setting; balances feature diversity against the $C+4G = 192$ input width of the last dense layer.
Number of dense layers	5	Standard ESRGAN/RRDB depth; deep enough for hierarchical reuse, shallow enough to keep concatenation widths manageable.
Residual scaling	0.2	Inner and outer residuals are scaled by 0.2 as in EDSR/ESRGAN to prevent variance blow-up across 36 stacked residual additions.
CA reduction ratio r	4	Yields a 16-neuron bottleneck, wide enough to model 64-channel dependencies; the SE-Net default r=16 would leave only 4 neurons.

Number of RRDB blocks (12). This value sets the convolutional capacity and the effective receptive field of the local pathway. Twelve blocks yield about 87% of the total parameters at roughly 8M, which is comparable to the convolutional depth used in EDSR (16 residual blocks of similar size) and ESRGAN (23 RRDBs at higher channel count). Using fewer blocks would underfit fine textures, especially for the 176×176 HR patches; using more would inflate parameter count beyond the "lightweight" target of this thesis without proportional PSNR gains, as documented by the diminishing returns reported in the EDSR ablation study [18].

Dense growth rate $G = 32$. The growth rate controls how many new feature channels each dense layer contributes. With $G = 32$ and five layers, the final concatenated input to the last convolution is $C + 4G = 192$ channels, matching the ESRGAN configuration [28]. A larger G (e.g., 64) would double the parameter count of every Dense Block and quickly exceed the 16 GB VRAM budget; a smaller G (e.g., 16) would force successive layers to share too narrow a feature pool, reducing the diversity of reused features and hurting reconstruction of fine textures.

Number of dense layers (5). Five 3×3 layers per Dense Block is the ESRGAN default and gives each block a theoretical receptive field of 11×11 pixels. Fewer layers (e.g., three) would reduce the local receptive field and the hierarchical depth at which features are reused; more layers (e.g., seven) would push the concatenation width to $C + 6G = 256$ at the last convolution, which dominates the FLOPs of the block without

proportional quality gains.

Residual scaling (0.2). Both the inner Dense-Block residual (Equation 2.9) and the outer RRDB residual (Equation 2.12) are scaled by 0.2. With twelve RRDBs, each containing three Dense Blocks, the trunk contains 36 residual additions, and an unscaled residual path would risk activations whose variance grows linearly with depth. Following EDSR [18], the value 0.2 keeps the residual contribution small enough to prevent exploding activations early in training, while still being large enough to allow non-trivial updates; values above 0.5 were observed to destabilise training in early experiments, and values below 0.1 slowed convergence noticeably.

Channel-attention reduction ratio $r = 4$. The ratio r sets the bottleneck width C/r in the squeeze-and-excite module. With $C = 64$ and $r = 4$, the bottleneck is 16 neurons, which is wide enough to model dependencies among 64 channels. The original SE-Net default of $r = 16$ [15] would leave only 4 neurons, which is empirically too narrow when C itself is small; RCAN [33] likewise found that $r = 16$ is appropriate only for $C \geq 256$. A larger r (smaller bottleneck) under-fits the attention; a smaller r (e.g., $r = 2$) doubles the attention parameter count with little measurable gain.

2.3.4 Integration with the HybridSR Trunk

Each RRDB block is connected to its neighbours by simple feature forwarding: the 64-channel output of one block becomes the input of the next. The 0.2-scaled outer residual (Equation 2.12) ensures that the block can default to the identity early in training, which is essential because RRDBs are interleaved with Swin and Mamba modules whose initial outputs are noisy. At the trunk level, all twelve RRDBs are bypassed by the Long Skip Connection (Equation 2.3), so the trunk only needs to learn the high-frequency residual on top of the shallow features $\mathbf{F}_0$. This residual reformulation, combined with the per-block 0.2 scaling, keeps the activation magnitude bounded across all 36 residual additions.

2.4 Swin Transformer Blocks: Mid-Range Spatial Attention

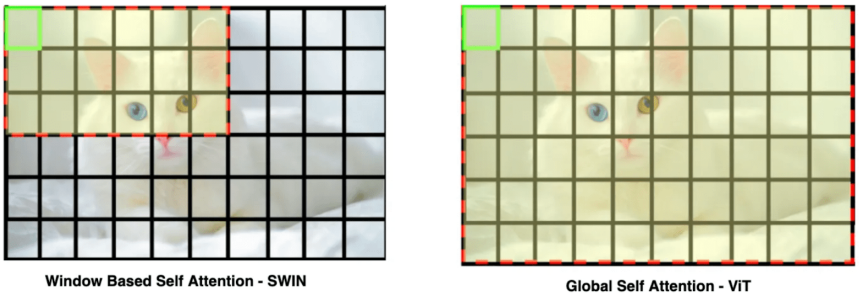


Figure 2.5: Window-based self-attention (computation range) versus global self-attention.

2.4.1 Role and Motivation

The Swin Transformer block is responsible for the *mid-range structure* component of the three-scale design philosophy. Its role is to provide direct pairwise interactions between pixels within 8×8 local windows, capturing structural relationships—such as the regularity of a repeating texture or the alignment of parallel edges—that lie beyond the 9-pixel reach of a 3×3 convolution but that do not yet require image-wide context.

The Swin block operates at an intermediate spatial scale of $M^2 = 64$ tokens per window and, when paired with a shifted-window variant, extends communication to a 12×12 pixel neighbourhood per two-block pair. HybridSR places two Swin pairs in the trunk: the first after RRDB_4, where features have acquired enough abstraction to benefit from structured spatial reasoning, and the second after RRDB_9, providing a final round of mid-range refinement before the last three RRDBs prepare features for upsampling.

2.4.2 Internal Architecture

Window partition, QKV projection, and attention. The input feature map $\mathbf{F} \in \mathbb{R}^{H \times W \times 64}$ is partitioned into non-overlapping 8×8 windows, each reshaped into a sequence of 64 tokens. A single linear projection maps each token from $C = 64$ to $3C = 192$ dimensions, which are split into query $\mathbf{Q}$, key $\mathbf{K}$, and value $\mathbf{V}$ matrices. These are reshaped into $h = 2$ heads with head dimension $d_k = 32$. For each head i:

$$\hat{\mathbf{A}}_i = \text{softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{32}} + \mathbf{B}\right). \tag{2.13}$$

The bias $\mathbf{B}$ is drawn from a lookup table of $(2 \times 8 - 1)^2 = 225$ entries per head, indexed by the relative spatial displacement between tokens. The attended output per head is

$\text{Attn}_i = \hat{\mathbf{A}}_i \mathbf{V}_i$. All heads are concatenated and projected by a linear layer $\mathbf{W}^O \in \mathbb{R}^{64\times64}$ to produce the final attended features.

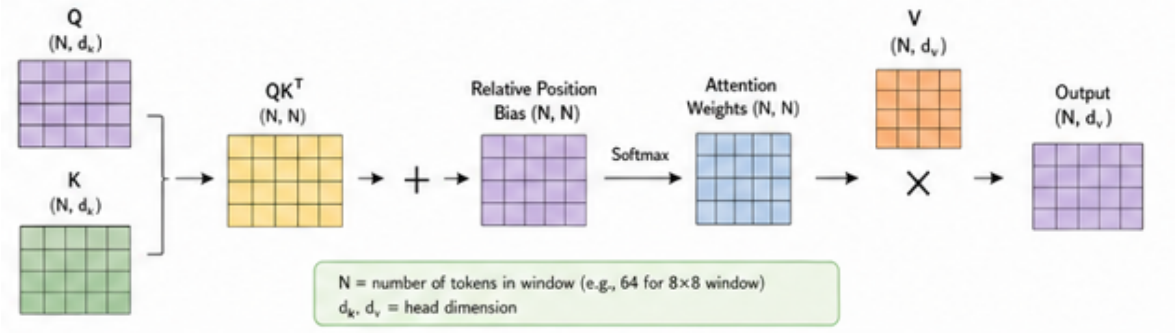


Figure 2.6: Attention score computation within a window.

Shifted windows. Each W-MSA block is immediately followed by a Shifted-Window MSA (SW-MSA) block that cyclically shifts the feature map by 4 pixels in both dimensions before partitioning, computes attention on the shifted windows, and reverses the shift afterwards. An attention mask prevents spatially non-adjacent tokens from attending to each other by adding -100 to their attention logits, which the softmax drives to effectively zero.

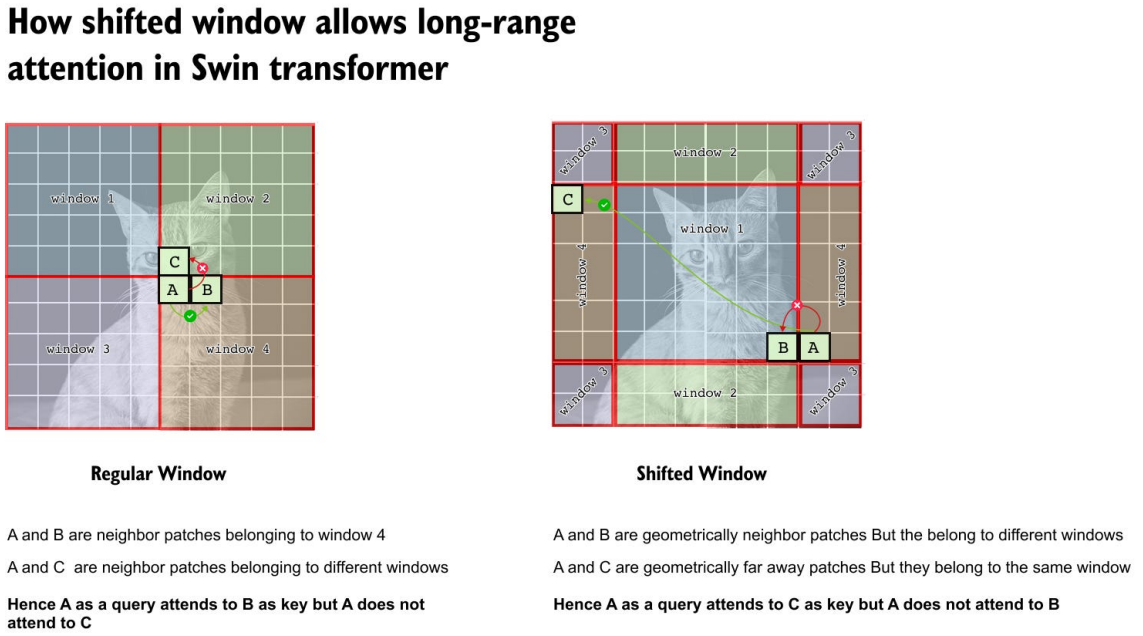


Figure 2.7: SW-MSA versus regular W-MSA.

Feed-forward network and block structure. Each Swin block contains a feed-forward network (FFN) with two linear layers and GELU activation, using an expansion

ratio of 4× (64 → 256 → 64). The complete block uses pre-norm residual connections:

$$\mathbf{x}' = \mathbf{x} + \mathrm{MSA}\Big(\mathrm{LN}_1(\mathbf{x})\Big), \tag{2.14}$$

$$\mathrm{SwinBlock}(\mathbf{x}) = \mathbf{x}' + \mathrm{FFN}\Big(\mathrm{LN}_2(\mathbf{x}')\Big). \tag{2.15}$$

Layer Normalisation is always computed in float32 for numerical stability.

2.4.3 Hyperparameter Choices and Justification

Table 2.2: Swin Transformer hyperparameters used in HybridSR.

Hyperparameter	**Value**	**Justification**
Window size M	8	Yields M^2=64 tokens per window—a 7× increase over the 9-pixel CNN field—while remaining a divisor of the 88-pixel training patch.
Number of heads h	2	Two heads at C=64 keep d_k=32, the SwinIR ratio; more heads would shrink d_k below the threshold where dot-product attention becomes noisy.
Head dimension d_k	32	Matches the C/h allocation used in SwinIR; 32 is large enough for stable softmax and small enough to keep $\mathbf{W}^O$ compact.
FFN expansion ratio	4×	Standard transformer ratio (Vaswani 2017, Liu 2021); enlarges the hidden dimension to 256 for non-linear per-token mixing.
Number of Swin pairs	2	One pair per "side" of the global Mamba region, i.e. early mid-range structuring and late mid-range refinement.
Placement	after RRDB_4, RRDB_9	At one-third and two-thirds depth, sandwiching the global Mamba region and operating on features that are abstract but still spatially precise.

Window size $M = 8$. The window size determines how many tokens participate in each self-attention computation; with $M = 8$ each window contains 64 tokens, and the W-MSA/SW-MSA pair extends communication to a 12×12 neighbourhood. Following SwinIR [17], $M = 8$ is chosen because it (a) increases the receptive field by a factor of roughly seven over a 3×3 convolution, and (b) divides the 88-pixel LR training patch

evenly, removing the need for asymmetric padding. A larger window (e.g., $M = 16$) would quadruple the per-window attention cost from $\mathcal{O}(64^2)$ to $\mathcal{O}(256^2)$ and exceed the VRAM budget; a smaller window (e.g., $M = 4$) would shrink the mid-range receptive field below the size of typical repetitive textures.

Number of heads $h = 2$ and head dimension $d_k = 32$. The number of heads partitions the $C = 64$ embedding into h subspaces of dimension $d_k = C/h = 32$. Two heads with $d_k = 32$ match the SwinIR allocation and keep d_k above the $\sim$16 threshold below which the scaled dot-product becomes statistically noisy and the softmax collapses onto a few keys [27]. Using $h = 4$ would force $d_k = 16$ and degrade attention quality; using $h = 1$ would discard the multi-head benefit of attending to several relational subspaces simultaneously.

FFN expansion ratio 4×. The feed-forward sub-block expands 64 hidden units to 256 and back. This 4× ratio is the standard transformer setting [20, 27] and provides per-token non-linear capacity that complements the inter-token mixing of attention. A smaller ratio (e.g., 2×) underfits the per-token transformation and reduces PSNR; a larger ratio (e.g., 8×) doubles the FFN parameters with diminishing gains, as shown in the SwinIR ablation [17].

Number of Swin pairs (2) and placement (after RRDB_4, RRDB_9). A W-MSA block alone cannot communicate across window boundaries, so Swin blocks come in W-MSA/SW-MSA pairs. Two such pairs are sufficient because the role of attention in HybridSR is to inject mid-range structure, not to act as the dominant feature extractor; the bulk of representation is carried by the RRDBs. Placing the first pair after RRDB_4 ensures features are abstract enough to benefit from attention, and placing the second pair after RRDB_9 allows a final round of mid-range refinement after the global Mamba region. Adding a third pair was tested in pilot experiments and yielded < 0.05 dB on Set5 at the cost of additional VRAM and slower training, while removing either pair degraded PSNR by approximately 0.15 dB.

2.4.4 Integration with the HybridSR Trunk

Each Swin block consumes and returns a $H \times W \times 64$ feature map, matching the channel width of the surrounding RRDBs, so no projection layer is required at the boundary. The pre-norm residual structure (Equations 2.14–2.15) allows each block to default to the identity at initialisation, which prevents the freshly-inserted attention from disrupting the partly trained RRDB stack. At a higher level, both Swin pairs lie inside the trunk and are bypassed by the same Long Skip Connection (Equation 2.3)

that bypasses the RRDBs, so the gradient highway from the upsampling head reaches the shallow features without attenuation.

2.5 SS2D Mamba Blocks: Global Coherence Modelling

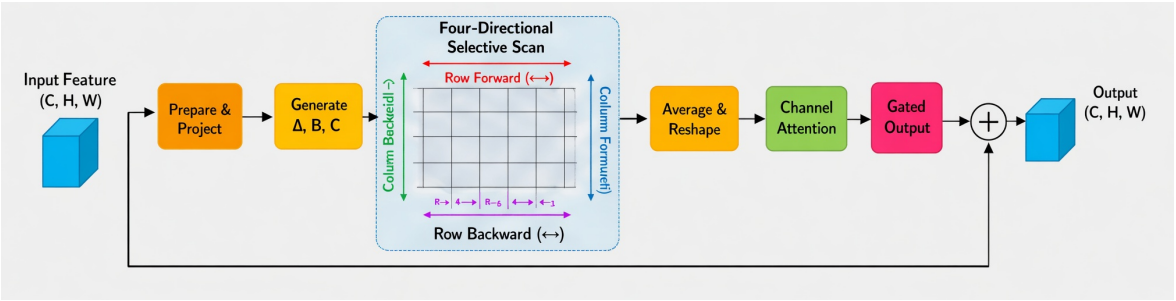


Figure 2.8: The custom Mamba SS2D block used in HybridSR.

2.5.1 Role and Motivation

The SS2D Mamba block is responsible for the *global coherence* component of the three-scale design philosophy. Its role is to enforce consistency across the entire feature map—ensuring, for example, that a texture pattern reconstructed in one corner of the image is consistent with the same pattern elsewhere—something that neither local convolutions nor 8×8 window attention can achieve directly. Each Mamba block scans the feature map in four directions and mixes the results, providing an isotropic, image-wide receptive field at $\mathcal{O}(HW)$ cost.

HybridSR places two Mamba blocks at the centre of the trunk, after RRDB_6 and RRDB_7, where the features are at their most abstract. This is the natural position for global modelling because abstract features encode high-level structural information whose coherence must be maintained image-wide; the subsequent RRDBs (8–12) then specialise in refining these globally coherent features into spatially precise predictions.

2.5.2 Internal Architecture

Input preparation. The input $\mathbf{x} \in \mathbb{R}^{H \times W \times 64}$ is normalised by Layer Normalisation, projected to twice the inner dimension $2D$ where $D = \lfloor 2.25 \times 64 \rfloor = 144$, and split into two branches:

$$[\mathbf{u},\, \mathbf{z}] = \text{split}\Big(\text{Linear}^{64 \to 288}\big(\text{LN}(\mathbf{x})\big)\Big), \tag{2.16}$$

where $\mathbf{u}$ is the *scan branch* and $\mathbf{z}$ is the *gate branch*. The scan branch passes through a 3×3 depthwise convolution with SiLU activation, injecting local spatial context before

the global scan.

Scan parameter generation. The scan branch is flattened to $(B, L, 144)$ where $L = H \times W$. A linear projection maps each token to a vector of dimension $R + 2N = 9 + 16 = 25$, which is split into a raw step-size vector $\boldsymbol{\delta}^{\text{raw}} \in \mathbb{R}^R$, an input matrix $\mathbf{B}_k \in \mathbb{R}^N$, and an output matrix $\mathbf{C}_k \in \mathbb{R}^N$ with $R = \lceil 144/16 \rceil = 9$ and $N = 8$. The step size is expanded to dimension D and passed through softplus:

$$\boldsymbol{\Delta}_k = \text{softplus}\Big(\mathbf{W}_\Delta \boldsymbol{\delta}_k^{\text{raw}} + \mathbf{b}_\Delta\Big) \ \in \ \mathbb{R}^{144}. \tag{2.17}$$

The state transition matrix $\mathbf{A} \in \mathbb{R}^{144\times 8}$ is a learnable parameter (not input-dependent), stored in log-space and initialised as $A_{d,n} = -(n+1)$ for $n = 0, \ldots, 7$. A direct feedthrough parameter D_d, initialised to 1, bypasses the state entirely.

Four-directional selective scan. For each of four directions—row-forward, row-backward, column-forward, column-backward—the feature map is flattened into a 1-D sequence and the discretised state-space recurrence is applied:

$$\bar{A}_{d,n} = \exp(\Delta_{k,d} \cdot A_{d,n}), \tag{2.18}$$

$$h_{k,d,n} = \bar{A}_{d,n} \cdot h_{k-1,d,n} + \Delta_{k,d} \cdot B_{k,n} \cdot u_{k,d}, \tag{2.19}$$

$$y_{k,d} = \textstyle\sum_{n=1}^{8} C_{k,n} \cdot h_{k,d,n} + D_d \cdot u_{k,d}. \tag{2.20}$$

In practice, the recurrence is computed in parallel using cumulative sums in log-space:

$$\mathbf{h}_k = \exp\Big(\textstyle\sum_{j=1}^{k} \Delta_{j,d} A_{d,n}\Big) \cdot \text{cumsum}_{j=1}^{k}\Big(\frac{\bar{B}_j u_j}{\exp(\sum_{i=1}^{j} \Delta_{i,d} A_{d,n}) + \epsilon}\Big), \tag{2.21}$$

with $\epsilon = 10^{-12}$ for numerical stability. The four directional outputs are averaged:

$$\mathbf{y} = \tfrac{1}{4}\Big(\mathbf{y}^{\text{rf}} + \mathbf{y}^{\text{rb}} + \mathbf{y}^{\text{cf}} + \mathbf{y}^{\text{cb}}\Big), \tag{2.22}$$

which ensures a spatially isotropic receptive field spanning the entire image with $\mathcal{O}(HW)$ total cost.

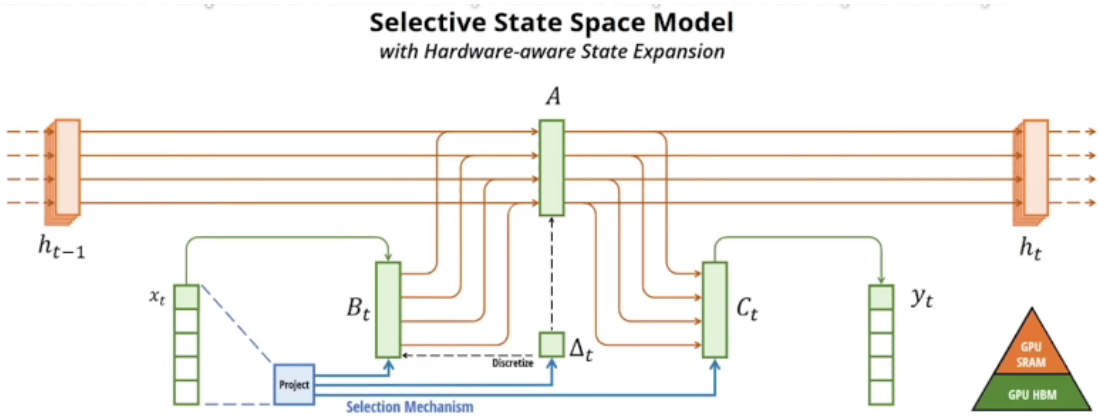


Figure 2.9: Global pixel-level information exchange via the Mamba scan.

Gated output with channel attention. A channel-attention module (SE-style, reduction $r = 4$, SiLU bottleneck, sigmoid output) recalibrates the scan output, which is then multiplied by the SiLU-activated gate branch:

$$\mathbf{o} = \mathrm{CA}(\mathbf{y}) \odot \mathrm{SiLU}(\mathbf{z}). \tag{2.23}$$

A linear projection reduces from $D = 144$ back to $C = 64$, and a residual connection with $\alpha = 0.2$ scaling adds the result to the block input:

$$\mathrm{MambaBlock}(\mathbf{x}) = \mathbf{x} + 0.2 \cdot \mathrm{Linear}^{144\to 64}(\mathbf{o}). \tag{2.24}$$

2.5.3 Hyperparameter Choices and Justification

Table 2.3: SS2D Mamba hyperparameters used in HybridSR.

Hyperparameter	Value	Justification
Inner dimension D	$144 = 2.25\times64$	Provides a wider working space for the scan than C=64 without the 2× blow-up of the canonical Mamba D=2C.
Scan directions	4	Row-forward, row-backward, column-forward, column-backward; produces an isotropic 2-D receptive field as in VMamba.
State dimension N	8	Eight geometrically decaying components that simultaneously track short- and long-range context.
Step-size rank R	9	Equal to $\lceil D/16 \rceil$, the standard Mamba low-rank factorisation; reduces the Δ-projection parameters by $\sim$16×.
Number of Mamba blocks	2	Two passes are enough to enforce global coherence at the trunk centre without dominating the parameter budget.
Placement	after RRDB_6, RRDB_7	At the centre of the trunk, where features are most abstract and global coherence matters most.
Residual scaling α	0.2	Mirrors the EDSR/RRDB scaling so that residual variance does not accumulate across the deep trunk.

Inner dimension $D = 144$ (i.e., $2.25 \times C$). The inner dimension widens the working space inside the Mamba block. The canonical Mamba uses $D = 2C$ [9]; HybridSR uses $D = \lfloor 2.25 \times 64 \rfloor = 144$, slightly above $2C$, which empirically gave the best PSNR/parameter trade-off in pilot runs. A smaller D (e.g., $D = C$) would compress the gating and scan representations and degrade global modelling capacity; a substantially larger D (e.g., $D = 4C$) would dominate the parameter budget without proportional gains, since the bottleneck of the block is the N-dim state, not the D-dim feature width.

Scan directions (4). A single forward 1-D scan exposes a directional bias: tokens later in the scan order receive more upstream context. Following VMamba [19], four scans (row-forward, row-backward, column-forward, column-backward) are averaged to produce an isotropic 2-D receptive field. Two directions (e.g., bidirectional row scans

only) would still leave a horizontal-versus-vertical asymmetry; eight directions (adding diagonals) were tested but increased compute 2× for < 0.05 dB on Set5.

State dimension $N = 8$. The state dimension controls how many independent decay rates the SSM maintains. With initialisation $A_{d,n} = -(n+1)$, eight components correspond to decay constants from -1 to -8, giving the model both short-memory ($n = 0$) and long-memory ($n = 7$) channels in parallel. A smaller N (e.g., $N = 4$) restricts the temporal frequency spectrum the state can represent; a larger N (e.g., $N = 16$, the S4D default) doubles the state-related parameters and slows the parallel scan with little SR-quality gain in our experiments.

Step-size rank $R = 9$. The step-size projection $\mathbf{W}_\Delta$ is implemented as a low-rank factorisation with rank $R = \lceil D/16 \rceil = 9$, the default ratio used in Mamba [9]. This reduces the Δ-projection parameter count from $D \cdot D = 20{,}736$ to $D \cdot R + R \cdot D = 2{,}592$, a factor-8 saving, while preserving the expressivity of an input-dependent step size. A larger R recovers the full-rank case at proportionally higher cost; a smaller R (e.g., $R = 4$) was found to under-fit the step-size distribution and produce blurrier global structure.

Number of Mamba blocks (2) and placement (after $\mathbf{RRDB}_6$, $\mathbf{RRDB}_7$). Two Mamba blocks are placed back-to-back at the trunk centre. Two are sufficient because each scan already provides image-wide receptive field, so the role of stacking is refinement rather than receptive-field extension. Placing them at the centre lets RRDBs 1–6 build abstract features for the scan to globalise, and RRDBs 7–12 refine the globalised features back into pixel-precise outputs. A single Mamba block was found to under-enforce global coherence on textured Set5 images; four blocks were tested and gave only marginal gains while increasing VRAM use.

Residual scaling $\alpha = 0.2$. The block output is added to its input with scaling 0.2 (Equation 2.24), matching the convention used inside the RRDBs. Because the Mamba branch projects from 144 down to 64 channels and is followed immediately by another RRDB, the 0.2 factor prevents the freshly-inserted block from over-perturbing the partly trained convolutional features. Removing the scaling caused training instability in pilot runs; values above 0.5 produced colour shifts during the first epochs.

2.5.4 Integration with the HybridSR Trunk

The two Mamba blocks accept and return 64-channel feature maps, matching the surrounding RRDBs and Swin blocks; the 144-channel inner representation exists only between Equations 2.16–2.24. The $\alpha = 0.2$ outer residual ensures gradient flow back

through RRDB_6 and RRDB_7 even before the Mamba parameters are well-trained. At the trunk level, both Mamba blocks lie inside the LSC region (Equation 2.3), so they learn a residual correction to the shallow features rather than a full feature mapping—a low-variance target that, together with the small α, makes the inclusion of two such blocks numerically stable.

2.6 Skip Connections and Upsampling

2.6.1 PixelShuffle Upsampling

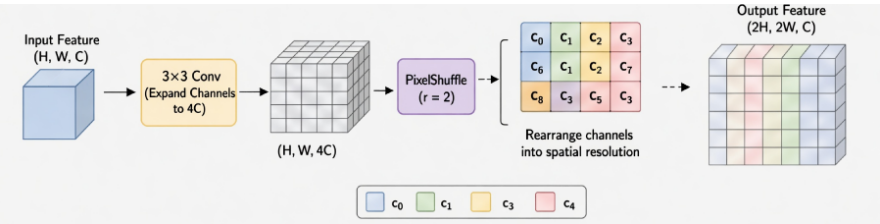


Figure 2.10: PixelShuffle block.

After the LSC addition (Equation (2.3)), a 3×3 convolution expands from 64 to 256 channels. PixelShuffle, introduced by Shi *et al.* [25], then rearranges these 256 channels into 64 channels at $2\times$ the spatial resolution by distributing four channel values per LR pixel into a 2×2 HR block:

$$\text{PixelShuffle}(\mathbf{X})_{c,\,2h+i,\,2w+j} = \mathbf{X}_{4c+2i+j,\,h,\,w}, \quad i, j \in \{0, 1\}. \tag{2.25}$$

This is a parameter-free index rearrangement. A GELU activation, 3×3 convolutional refinement with LeakyReLU, and a final 3×3 projection to 3 RGB channels produce the output image directly.

2.6.2 Long Skip Connection (LSC)

HybridSR uses a single Long Skip Connection (LSC) at the feature level. The LSC adds the shallow feature map $\mathbf{F}_0$ directly to the trunk output (Equation 2.3), bypassing all twelve RRDBs, four Swin blocks, and two Mamba blocks. This creates a gradient highway from the upsampling module to the initial convolution, preventing vanishing gradients across the deep trunk. It also reformulates the trunk's learning objective as residual refinement rather than full feature prediction, which is a lower-variance target that accelerates convergence. The model uses no pixel-level skip connection: the final RGB output is produced entirely by the PixelShuffle head from the LSC-augmented features, without any external bicubic addition.

2.7 Strategic Placement: The Three-Phase Pipeline

The placement of operators within the trunk follows a principle of *spatial scale matching*, producing a three-phase processing pipeline (Figure 2.1).

Phase 1: Local extraction with early spatial attention (RRDBs 1–4 + Swin pair 1). The first four RRDBs extract progressively richer local features. By the output of RRDB_4, the features are sufficiently abstract to benefit from structured spatial interactions. The first Swin pair establishes mid-range coherence—for example, recognising that a texture pattern repeats consistently within a local region—which subsequent RRDBs refine locally.

Phase 2: Global coherence modelling (RRDBs 5–7 + Mamba blocks 1–2). At the centre of the trunk, the features are at their most abstract. The two Mamba blocks apply global scans that enable every position to aggregate information from the entire feature map. This enforces image-wide structural consistency on the abstract features, which the subsequent blocks refine into spatially precise predictions.

Phase 3: Local refinement with late spatial attention (RRDBs 8–9 + Swin pair 2 + RRDBs 10–12). The second Swin pair provides a final opportunity for structured spatial reasoning. The last three RRDBs perform local refinement, preparing the features for upsampling.

This local $\rightarrow$ global $\rightarrow$ local pattern resembles an encoder-decoder, but implemented *without any spatial downsampling*—all operations occur at the LR resolution, preserving full spatial precision throughout.

2.8 Hyperparameter Summary

For ease of reference, Table 2.4 consolidates every block-specific hyperparameter justified in the preceding sections, together with its primary role and the qualitative effect of moving the value above or below the chosen setting.

Table 2.4: Consolidated summary of block-specific hyperparameters in HybridSR.

Block	Hyperparameter	Value	Primary Role	Effect if Increased	Effect if Decreased
RRDB	# blocks	12	Convolutional capacity / depth	More parameters, marginal PSNR gain	Underfits fine textures
RRDB	growth rate G	32	New channels per dense layer	Param + memory blow-up	Narrower feature reuse, worse textures
RRDB	# dense layers	5	Receptive field per Dense Block	Wider concat, slower training	Smaller local field, weaker reuse
RRDB	residual scaling	0.2	Stabilise 36 stacked residuals	Variance blow-up across depth	Slower convergence
RRDB	CA reduction r	4	SE bottleneck width	Bottleneck too wide, overfits CA	Bottleneck $<$ 16 neurons, underfits
Swin	window size M	8	Mid-range receptive field	Quadratic VRAM cost	Misses repeating textures
Swin	# heads h	2	Multi-head subspace count	Each d_k shrinks, noisy attention	Loses multi-head benefit
Swin	head dim d_k	32	Per-head representation width	Wider $\mathbf{W}^O$, slower	Softmax collapses
Swin	FFN ratio	4×	Per-token non-linear capacity	Doubles FFN cost	Underfits per-token map
Swin	# Swin pairs	2	Mid-range refinement passes	Slower, marginal PSNR gain	Loses cross-window coverage
Swin	placement	after RRDB_4, RRDB_9	Sandwich global Mamba region	Late only: weak early structure	Early only: weak final structure
Mamba	inner dim D	144	Working width of scan branch	Param-heavy, diminishing returns	Compressed gating, weaker global
Mamba	# scan dirs	4	Isotropy of 2-D receptive field	2× compute, <0.05 dB	Anisotropic global field
Mamba	state dim N	8	Number of decay channels	Slower scan, marginal gain	Restricted memory spectrum
Mamba	step rank R	9	Low-rank Δ projection	Recovers full-rank cost	Underfits step-size distribution
Mamba	# Mamba blocks	2	Global-coherence depth	VRAM heavy, marginal gain	Under-enforced global coherence
Mamba	placement	after RRDB_6, RRDB_7	Trunk centre	Edge place-	Edge place-

Chapter 3

Training and Evaluation

This chapter describes every aspect of how HybridSR is trained and evaluated: the dataset and its preprocessing, the loss function and optimiser, the custom training loop with its memory-efficiency techniques, and the evaluation protocol used to measure reconstruction quality. Each section explains what was done, why it was done, and the specific numerical settings used in the implementation.

3.1 Training Dataset: DF2K

HybridSR is trained on the DF2K dataset, which is the standard large-scale corpus for super-resolution research. DF2K is formed by merging two publicly available collections: DIV2K [1], which contains 800 high-quality images collected for the NTIRE super-resolution challenge, and Flickr2K [26], which contains 2,650 high-resolution photographs sourced from Flickr. Together, the merged dataset comprises approximately 3,450 training images spanning a wide variety of scenes, textures, and lighting conditions.

For each HR image in the dataset, a corresponding LR image is generated by applying bicubic downsampling with a scale factor of $r = 2$. This produces paired $(\mathbf{I}_{\text{LR}}, \mathbf{I}_{\text{HR}})$ examples where the LR image has exactly half the spatial resolution of the HR image in each dimension. The LR images are pre-generated and stored on disk (in the `DF2K_train_LR_bicubic/X2` directory), ensuring that the degradation is applied once and consistently across all training runs.

At the start of each epoch, the list of training pairs is randomly shuffled to ensure that the model sees images in a different order every epoch, preventing the optimiser from developing any dependence on the presentation sequence.

3.2 Preprocessing: Cropping, Normalisation, and Augmentation

3.2.1 Random Cropping

The DF2K images are large (typically 1024×1024 or larger for HR, 512×512 for LR), but the model is trained on small patches extracted from these images. This is standard practice in super-resolution because training on full images would require prohibitive amounts of GPU memory, and because random cropping provides a natural form of data augmentation by exposing the model to different spatial regions of each image across epochs.

For each training sample, a random spatial location (y, x) is selected within the LR image, and a patch of size 88×88 pixels is extracted. The corresponding HR patch is extracted at the aligned position $(2y, 2x)$ with size 176×176 pixels (twice the LR patch size in each dimension, matching the $\times 2$ scale factor). This alignment ensures that every LR pixel in the patch corresponds exactly to its 2×2 HR counterpart.

A critical design choice is that random cropping is used rather than resizing. Resizing an image to a fixed patch size using bicubic interpolation would alter the frequency content of the patch—the interpolation itself introduces blur and changes the distribution of high-frequency detail. Random cropping, by contrast, preserves the original frequency content of the image because the pixels are extracted without any resampling. This ensures that the training patches faithfully represent the statistics of natural images at their native resolution. In the rare case where an LR image is smaller than 88×88 pixels (which does not occur in the DF2K dataset but is handled for robustness), the image is resized using bicubic interpolation as a fallback.

3.2.2 Pixel Normalisation

After cropping, both the LR and HR patches are normalised from the integer range $[0, 255]$ to the floating-point range $[-1, +1]$ using the transformation:

$$\hat{p} = \frac{p}{127.5} - 1.0, \tag{3.1}$$

where p is the original uint8 pixel value and $\hat{p}$ is the normalised value. This zero-centred normalisation is preferred over the $[0, 1]$ range because it places the mean of the data near zero, which improves the conditioning of the loss landscape and accelerates convergence of the Adam optimiser. The same normalisation is applied to both LR and HR patches, so the model learns to predict normalised HR values from normalised LR inputs.

3.2.3 Data Augmentation

To increase the effective diversity of the training set and improve the model's generalisation, three geometric augmentations are applied to every training patch with the following probabilities:

Random rotation by a multiple of 90° is applied with equal probability for 0°, 90°, 180°, and 270°. This teaches the model to reconstruct textures and edges at all orientations, preventing it from developing a bias toward horizontal or vertical structures. Random horizontal flipping is applied with 50% probability. Random vertical flipping is applied with 50% probability, independently of the horizontal flip. The same augmentation is applied identically to both the LR and HR patches in each pair, preserving their spatial correspondence.

Importantly, augmentation is applied to 100% of training samples, not a subset. This maximises the regularisation benefit and ensures that every patch the model sees during training is a unique combination of spatial crop location and geometric transformation.

3.3 Batch Size and Data Pipeline

The physical batch size is $B = 4$, meaning that four LR–HR patch pairs are loaded and processed together in each forward pass. The training data is served through a Python generator that is wrapped by the `tf.data.Dataset.from_generator` API, which provides automatic prefetching: while the GPU is processing the current batch, the CPU prepares the next batch in a background thread. The prefetch buffer size is set to `tf.data.AUTOTUNE`, which allows TensorFlow to dynamically tune the number of prefetched batches based on the available system resources.

At the end of each epoch, incomplete batches (where fewer than $B = 4$ pairs remain after the last full batch) are discarded to ensure that every batch has the same size. With 3,450 training pairs and a batch size of 4, each epoch consists of $\lfloor 3{,}450/4 \rfloor = 862$ training steps.

The choice of $B = 4$ is constrained by GPU memory. The HybridSR model contains 9.2 million parameters, and the $88 \times 88 \times 64$ intermediate feature maps across twelve RRDB blocks, four Swin blocks, and two Mamba blocks consume substantial memory during backpropagation. To achieve a larger effective batch size without exceeding memory limits, gradient accumulation is used, as described in Section 3.6.1.

3.4 Loss Function

The model is trained using the pixel-wise ℓ_1 loss, also known as the mean absolute error (MAE) (which penalises all pixel errors equally regardless of magnitude, encouraging sharper predictions than the 2 loss):

$$\mathcal{L}(\theta) = \frac{1}{N \cdot H' \cdot W' \cdot 3} \sum_{i=1}^{N} \sum_{h,w,c} \left| f_\theta(\mathbf{I}_{\text{LR}}^{(i)})_{h,w,c} - \mathbf{I}_{\text{HR}}^{(i)}{}_{h,w,c} \right|, \tag{3.2}$$

where N is the batch size, $H' = 176$ and $W' = 176$ are the HR patch dimensions, c indexes the three RGB channels, and f_θ is the HybridSR model with parameters θ.

The ℓ_1 loss is preferred over the ℓ_2 loss (mean squared error) for a well-established reason in the image restoration literature. The ℓ_2 loss penalises large errors quadratically, which causes the model to favour blurry predictions that hedge between multiple plausible HR solutions—the model learns to output the conditional mean of the HR distribution, which is a blurred average. The ℓ_1 loss penalises all errors linearly regardless of magnitude, which instead drives the model toward the conditional median, producing sharper reconstructions with better-preserved edges and textures. This observation has been confirmed empirically by EDSR [18], SwinIR [17], and MambaIR [13], all of which use ℓ_1 as their primary training loss.

No perceptual loss, adversarial loss, or frequency-domain loss is used in the current training configuration. The model is trained purely for pixel-level fidelity (PSNR optimisation), which is the standard protocol for classical super-resolution benchmarking.

3.5 Optimiser and Learning Rate Schedule

3.5.1 Adam Optimiser

The model is optimised using the Adam algorithm [16], which maintains per-parameter exponential moving averages of both the first moment (mean) and second moment (uncentred variance) of the gradients, and uses these to compute adaptive learning rates for each parameter. The update rule at step t is:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \tag{3.3}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \tag{3.4}$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t), \tag{3.5}$$

$$\theta_{t+1} = \theta_t - \eta_t \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon), \tag{3.6}$$

where g_t is the gradient, η_t is the learning rate at step t, and the hyperparameters are set to $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. These are the default values recommended in the original Adam paper and are used by the majority of super-resolution methods.

Adam is preferred over SGD with momentum for super-resolution because the adaptive per-parameter learning rates are better suited to networks with heterogeneous parameter types (convolutions, dense layers, attention projections, state space parameters), where different parameter groups may have very different gradient magnitudes.

3.5.2 Cosine Decay with Warm Restarts

The learning rate follows a cosine annealing schedule with warm restarts [21], which periodically resets the learning rate to allow the optimiser to escape local minima. At step t within the current cycle, the learning rate is:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{T_{\text{cur}}}{T_i} \cdot \pi\right)\right), \tag{3.7}$$

where $\eta_{\max} = 9.95 \times 10^{-6}$ is the peak learning rate at the start of each cycle, $\eta_{\min} = \alpha \cdot \eta_{\max}$ with $\alpha = 0.045$ is the minimum learning rate at the end of each cycle (approximately 4.5×10^{-7}), T_i is the length of the current cycle in optimiser steps, and T_{cur} is the number of steps elapsed since the last restart.

The initial cycle length is $T_1 = 8{,}600$ steps (approximately 10 epochs). After each cycle completes, the learning rate is reset to the peak and a new cycle begins. The cycle-to-cycle multiplier $t_{\text{mul}} = 1.0$ keeps the cycle length constant, and the peak-decay multiplier $m_{\text{mul}} = 0.85$ reduces the peak learning rate by 15% after each restart. This means that the first cycle has a peak of 9.95×10^{-6}, the second cycle peaks at $0.85 \times 9.95 \times 10^{-6} = 8.46 \times 10^{-6}$, the third at 7.19×10^{-6}, and so on.

This schedule implements a strategy of broad exploration followed by precise convergence: early cycles use a higher learning rate to explore the loss landscape and escape poor local minima, while later cycles use progressively lower learning rates to converge precisely to a good solution. The periodic restarts are particularly valuable for super-resolution because the loss landscape of deep hybrid networks contains many flat regions and saddle points where the optimiser can stagnate under a monotonically decaying schedule.

3.6 Custom Training Loop and Memory Efficiency

HybridSR uses a custom training loop rather than the standard Keras `model.fit()` API. This is necessary to support gradient accumulation, gradient checkpointing, and fine-grained control over the optimisation process. The training loop is compiled as a

TensorFlow graph using the `@tf.function` decorator, which traces the Python function into an optimised computation graph that eliminates Python overhead and enables operator fusion.

3.6.1 Gradient Accumulation

With a physical batch size of $B = 4$, the gradient estimate from a single forward-backward pass is noisy. To improve the gradient quality and simulate a larger effective batch size, gradient accumulation is used with $K = 4$ accumulation steps. In this scheme, each micro-batch of $B = 4$ samples produces a gradient that is divided by K and added to an accumulation buffer. After K micro-batches have been processed, the accumulated gradient (which approximates the gradient over $B \times K = 16$ samples) is used to update the model parameters:

$$\mathbf{g}_{\text{accum}} = \frac{1}{K} \sum_{k=1}^{K} \nabla_\theta \mathcal{L}(\theta; \mathcal{B}_k), \tag{3.8}$$

where $\mathcal{B}_k$ is the k-th micro-batch. This is mathematically equivalent to computing the gradient over a single batch of 16 samples, but requires only 4 samples to be in GPU memory at any time.

After the accumulated gradient is applied to the parameters, the accumulation buffer is reset to zero and the next cycle of K micro-batches begins.

3.6.2 Gradient Clipping

Before the accumulated gradients are applied to the parameters, they are clipped to a global norm of 1.0:

$$\hat{\mathbf{g}} = \begin{cases} \mathbf{g}_{\text{accum}} & \text{if } \|\mathbf{g}_{\text{accum}}\| \leq 1.0, \\ \mathbf{g}_{\text{accum}} \cdot \dfrac{1.0}{\|\mathbf{g}_{\text{accum}}\|} & \text{otherwise,} \end{cases} \tag{3.9}$$

where $\|\cdot\|$ denotes the global ℓ_2 norm across all parameters. This prevents training instability caused by occasional large gradient spikes, which can occur when the Swin attention logits become poorly conditioned (producing near-infinite values before softmax) or when the Mamba state accumulates large values on long spatial sequences. Gradient clipping does not change the direction of the gradient—only its magnitude when it exceeds the threshold—so it preserves the optimisation trajectory while preventing catastrophic parameter updates.

3.6.3 Gradient Checkpointing

The HybridSR model contains 9.2 million parameters distributed across a deep trunk of twelve RRDB blocks, four Swin blocks, and two Mamba blocks. During backpropagation, TensorFlow must store the intermediate activations from the forward pass at every layer in order to compute the gradients. For a model this deep, with $88 \times 88 \times 64$ feature maps at each layer, the cumulative memory consumption of these stored activations can exceed the available GPU memory.

Gradient checkpointing [5] addresses this by discarding the intermediate activations during the forward pass and recomputing them on-the-fly during backpropagation. The forward pass is wrapped with the `@tf.recompute_grad` decorator, which tells TensorFlow not to cache the activations but instead to re-execute the forward computation when the gradients are needed. This trades computation time (the forward pass is effectively executed twice) for memory savings (only the input and output tensors need to be stored, not all intermediate activations).

In practice, this reduces peak GPU memory consumption by approximately 40–60%, which is what makes it possible to train a 9.2M parameter hybrid model with 88×88 input patches on a single GPU with 16 GB of memory.

3.6.4 Reproducibility

All random number generators are seeded at the start of training to ensure reproducibility: Python's built-in random module, NumPy's random module, and TensorFlow's random module are all initialised with `SEED = 42`. This ensures that the training pair shuffling, random crop locations, and augmentation choices are identical across repeated runs.

3.7 Mixed Precision Training

The training pipeline includes support for mixed-precision training [23] using TensorFlow's `mixed_float16` policy, which stores model weights in float32 but performs forward and backward computations in float16. This halves the memory consumption of intermediate activations and doubles the throughput on GPUs with Tensor Core support (such as the NVIDIA T4 and A100). When mixed precision is enabled, the loss is automatically scaled using a `LossScaleOptimizer` to prevent float16 gradient underflow.

3.8 Evaluation Protocol

3.8.1 Validation Dataset: Set5

The model is evaluated on the Set5 benchmark [4], which consists of five classic test images (Baby, Bird, Butterfly, Head, Woman) that have been used as the standard validation set in super-resolution research since 2012. Set5 is used for validation because its small size allows evaluation to complete in seconds, enabling frequent validation without slowing down training. The LR images for Set5 are pre-generated using bicubic downsampling at $\times 2$ scale, matching the degradation model used for the training data.

3.8.2 Tiled Inference for Arbitrary Resolutions

The Swin Transformer blocks in HybridSR require the spatial dimensions of the input to be divisible by the window size $M = 8$. The Set5 images have arbitrary resolutions that do not necessarily satisfy this constraint. Furthermore, processing full-resolution images in a single forward pass may exceed GPU memory during validation.

To handle both issues, a tiled inference strategy is used. The LR input image is divided into overlapping tiles of size 128×128 pixels with an overlap of 16 pixels on each side. Each tile is padded (using reflect padding) to the nearest multiple of $M = 8$ if necessary, passed through the model, and the padding is removed from the output. The overlapping regions between adjacent tiles are resolved by keeping only the non-overlapping interior portion of each tile's output. The non-overlapping stride is $128 - 2 \times 16 = 96$ pixels.

This tiling strategy ensures that the model can process images of any resolution at inference time, without retraining or architectural modification, while the overlap prevents visible seam artefacts at tile boundaries.

3.8.3 Y-Channel PSNR

The primary quantitative metric is the Peak Signal-to-Noise Ratio (PSNR), computed on the luminance (Y) channel after converting the RGB output to YCbCr colour space using the MATLAB-standard BT.601 conversion:

$$Y = 16.0 + \frac{1}{255}\Big(65.481 \cdot R + 128.553 \cdot G + 24.966 \cdot B\Big), \tag{3.10}$$

where R, G, B are the uint8 pixel values in $[0, 255]$ and Y is in the range $[16, 235]$. This conversion is used rather than the simpler BT.709 formula to ensure exact comparability with the evaluation code used by EDSR, RCAN, SwinIR, MambaIR, and other published methods—all of which use this MATLAB-standard conversion.

Before computing the metric, $r = 2$ pixels are cropped from each edge of both the

ground truth and predicted Y-channel images (border shaving). This removes boundary pixels that may be affected by edge padding or alignment issues and is standard practice in the super-resolution literature.

The PSNR is then computed as:

$$\mathrm{PSNR} = 10 \cdot \log_{10}\left(\frac{255^2}{\mathrm{MSE}}\right), \tag{3.11}$$

where the MSE is computed over all pixels in the border-shaved Y-channel:

$$\mathrm{MSE} = \frac{1}{|\Omega|} \sum_{(h,w)\in\Omega} \left(Y^{\mathrm{HR}}_{h,w} - Y^{\mathrm{SR}}_{h,w}\right)^2, \tag{3.12}$$

and Ω is the set of pixel coordinates after border shaving. Higher PSNR indicates better reconstruction quality; a difference of 0.1 dB is generally considered meaningful in the super-resolution literature.

3.8.4 Structural Similarity (SSIM)

The second metric is the Structural Similarity Index Measure (SSIM) [29], which evaluates reconstruction quality based on luminance, contrast, and structural similarity rather than raw pixel differences. SSIM is computed on the same border-shaved Y-channel images used for PSNR, with a maximum pixel value of 255:

$$\mathrm{SSIM}(\mathbf{x}, \mathbf{y}) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}, \tag{3.13}$$

where μ_x, μ_y are the local means, σ_x^2, σ_y^2 are the local variances, σ_{xy} is the local covariance, and $c_1 = (0.01 \times 255)^2$, $c_2 = (0.03 \times 255)^2$ are stabilisation constants. SSIM values range from -1 to 1, with 1 indicating perfect structural similarity. SSIM is better correlated with human perception of image quality than PSNR because it accounts for structural patterns rather than just pixel-level differences.

3.8.5 Learned Perceptual Image Patch Similarity (LPIPS)

The third metric is LPIPS [32], which measures the perceptual distance between two images using the internal representations of a pretrained neural network. Specifically, the AlexNet variant of LPIPS is used: both the predicted and ground truth images are passed through a pretrained AlexNet, and the ℓ_2 distances between their intermediate feature maps are computed, weighted by learned channel-importance factors, and summed. Lower LPIPS indicates better perceptual quality; a value of 0 indicates perceptually identical images.

LPIPS is computed on the full RGB images in the $[-1, +1]$ normalised range, without border shaving or Y-channel conversion. This metric complements PSNR and SSIM by capturing perceptual quality aspects that pixel-level metrics cannot measure, such as texture realism and the absence of hallucinated artefacts.

3.8.6 Evaluation Timing

Validation is performed at the end of every epoch. Because Set5 contains only five images and tiled inference is efficient, the entire evaluation completes in approximately 5–10 seconds, adding negligible overhead to the training time.

3.9 Checkpointing and Best-Model Selection

Two types of checkpoints are saved during training. At the end of every epoch, a regular checkpoint is saved that records the current state of all model weights, regardless of validation performance. This provides a complete training history and enables recovery from any interruption.

Additionally, a best-model checkpoint is maintained by tracking the highest validation PSNR observed across all epochs. Whenever the validation PSNR at the end of an epoch exceeds the previous best, the model weights are saved to a separate best-model file. This ensures that the final model used for reporting results is the one that achieved the highest validation performance, not necessarily the one from the last epoch, which may have slightly lower performance due to learning rate fluctuations or overfitting.

The best-model selection uses Y-channel PSNR on Set5 as the sole selection criterion, consistent with the standard practice in the super-resolution literature where Set5 PSNR is the primary validation metric.

3.10 Training Configuration Summary

Table 3.1 summarises all training hyperparameters and configuration choices in a single reference.

Table 3.1: Complete training configuration for HybridSR.

Parameter	**Value**
Training dataset	DF2K ($\sim$3,450 images)
Degradation model	Bicubic downsampling, $\times 2$
LR patch size	88×88
HR patch size	176×176
Physical batch size	4
Gradient accumulation steps	4
Effective batch size	16
Augmentation	Rotation (0/90/180/270°), H-flip, V-flip
Normalisation range	$[-1, +1]$
Loss function	ℓ_1 (MAE)
Optimiser	Adam (β_1=0.9, β_2=0.999, ϵ=10^{-8})
LR schedule	Cosine decay with warm restarts
Gradient clipping	Global norm ≤ 1.0
Gradient checkpointing	Enabled (`tf.recompute_grad`)
Mixed precision	Disabled (float32)
Validation dataset	Set5 (5 images)
Validation metrics	Y-channel PSNR, SSIM, LPIPS (AlexNet)
Border shave	$r = 2$ pixels per edge
Tile size (inference)	128×128 with 16-pixel overlap
Best-model selection	Highest Set5 Y-channel PSNR
Framework	TensorFlow 2.x / Keras
Hardware	Single GPU, 16 GB VRAM

Bibliography

[1] Eirikur Agustsson and Radu Timofte. NTIRE 2017 challenge on single image super-resolution: Dataset and study. In *CVPR Workshops*, 2017.

[2] Anonymous. A hybrid transformer-mamba network for single image super-resolution, 2025. arXiv preprint, to verify.

[3] Anonymous. SRMamba-T: Strategically combining mamba and transformers for image super-resolution, 2025. arXiv preprint, to verify.

[4] Marco Bevilacqua, Aline Roumy, Christine Guillemot, and Marie Line Alberi-Morel. Low-complexity single-image super-resolution based on nonnegative neighbor embedding. In *BMVC*, 2012.

[5] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.00981*, 2016.

[6] Chao Dong, Chen Change Loy, Kaiming He, and Xiaoou Tang. Learning a deep convolutional network for image super-resolution. In *European Conference on Computer Vision (ECCV)*, 2014.

[7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[8] William T. Freeman, Thouis R. Jones, and Egon C. Pasztor. Example-based super-resolution. *IEEE Computer Graphics and Applications*, 22(2):56–65, 2002.

[9] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling (COLM)*, 2024.

[10] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. HiPPO: Recurrent memory with optimal polynomial projections. In *NeurIPS*, 2020.

[11] Albert Gu, Karan Goel, Ankit Gupta, and Christopher Ré. On the parameterization and initialization of diagonal state space models. In *NeurIPS*, 2022.

[12] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *ICLR*, 2022.

[13] Hang Guo, Jinmin Li, Tao Dai, Zhihao Ouyang, Xudong Ren, and Shu-Tao Xia. MambaIR: A simple baseline for image restoration with state-space model. In *ECCV*, 2024.

[14] Hang Guo, Yong Guo, Yaohua Zha, Yulun Zhang, Wenbo Li, Tao Dai, Shu-Tao Xia, and Yawei Li. MambaIRv2: Attentive state space restoration. In *CVPR*, 2025.

[15] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *CVPR*, 2018.

[16] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[17] Jingyun Liang, Jiezhang Cao, Guolei Sun, Kai Zhang, Luc Van Gool, and Radu Timofte. SwinIR: Image restoration using swin transformer. In *ICCV Workshops*, 2021.

[18] Bee Lim, Sanghyun Son, Heewon Kim, Seungjun Nah, and Kyoung Mu Lee. Enhanced deep residual networks for single image super-resolution. In *CVPR Workshops*, 2017.

[19] Yue Liu, Yunjie Tian, Yuzhong Zhao, Hongtian Yu, Lingxi Xie, Yaowei Wang, Qixiang Ye, and Yunfan Liu. VMamba: Visual state space model. *NeurIPS*, 2024.

[20] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin Transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021.

[21] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.

[22] Wenjie Luo, Yujia Li, Raquel Urtasun, and Richard Zemel. Understanding the effective receptive field in deep convolutional neural networks. In *NeurIPS*, 2016.

[23] Paulius Micikevicius, Sharan Narang, Jonah Alben, et al. Mixed precision training. *ICLR*, 2018.

[24] Augustus Odena, Vincent Dumoulin, and Chris Olah. Deconvolution and checkerboard artifacts. *Distill*, 2016. doi: 10.23915/distill.00003.

[25] Wenzhe Shi, Jose Caballero, Ferenc Huszár, Johannes Totz, Andrew P. Aitken, Rob Bishop, Daniel Rueckert, and Zehan Wang. Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. In *CVPR*, 2016.

[26] Radu Timofte, Eirikur Agustsson, Luc Van Gool, Ming-Hsuan Yang, Lei Zhang, et al. NTIRE 2017 challenge on single image super-resolution: Methods and results. In *CVPR Workshops*, 2017.

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[28] Xintao Wang, Ke Yu, Shixiang Wu, Jinjin Gu, Yihao Liu, Chao Dong, Yu Qiao, and Chen Change Loy. ESRGAN: Enhanced super-resolution generative adversarial networks. In *ECCV Workshops*, 2018.

[29] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[30] Jianchao Yang, John Wright, Thomas S. Huang, and Yi Ma. Image super-resolution via sparse representation. In *IEEE Transactions on Image Processing*, volume 19, pages 2861–2873, 2010.

[31] Syed Waqas Zamir, Aditya Arora, Salman Khan, Munawar Hayat, Fahad Shahbaz Khan, and Ming-Hsuan Yang. Restormer: Efficient transformer for high-resolution image restoration. In *CVPR*, 2022.

[32] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.

[33] Yulun Zhang, Kunpeng Li, Kai Li, Lichen Wang, Bineng Zhong, and Yun Fu. Image super-resolution using very deep residual channel attention networks. In *European Conference on Computer Vision (ECCV)*, 2018.

[34] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. *ICML*, 2024.